Homi Hostel
The Complete Technical Manual

Every subsystem, every decision, every trap — and how to build it again from nothing.

A Flutter + Firebase hostel management system for SLIET, documented end to end.

Pravjeet Chaudhary

Contents

How to Read This Manual 6

What this covers, and what it deliberately does not . . . 6

How to use it . . . 6

Part I — The Problem and the Shape of the Answer 7

What the application is . . . 7

Who uses it . . . 7

The constraint that shaped the whole system . . . 7

The shape of the answer . . . 8

Part I — Why Flutter and Firebase 9

The choice . . . 9

What Flutter actually gives you here . . . 9

What Firebase gives you . . . 9

What it cost . . . 9

Part I — The One Architectural Rule 11

One dashboard, permission-filtered . . . 11

Why never branch on role names . . . 11

The excludes escape hatch . . . 11

The permission catalogue . . . 12

Deciding staff from resident, without asking . . . 12

Part II — Firebase From Zero 13

Create the project . . . 13

Enable Authentication . . . 13

Create the Firestore database . . . 13

Wire the app to the project . . . 13

Initialise Firebase in the app . . . 14

Deploy the rules . . . 14

Part II — The Data Model 15

The shape . . . 15

Why users is a root collection . . . 15

Deterministic document ids . . . 15

Denormalisation, and where it is deliberate . . . 15

Year belongs to the session, not the student . . . 16

The office order shape . . . 16

Part II — The Security Rules, Explained **17**

The governing idea . . . 17

The helper functions . . . 17

Super Admin short-circuits everything . . . 17

Tenant isolation, enforced per document . . . 18

Preventing privilege escalation . . . 18

Enrollments are append-only . . . 18

The default deny . . . 19

Aggregation queries obey the same rules . . . 19

Testing the rules . . . 19

Part III — The Codebase Layout **20**

The rule that keeps it navigable . . . 20

Drill-downs are state, not routes . . . 20

Part III — core/: The Four Pillars **21**

session.dart — who is looking at this screen . . . 21

identity.dart — logging in without an email . . . 22

theme.dart . . . 22

Part III — models/: The Data Shapes **24**

Computed properties belong on the model . . . 24

CollegeSettings — configuration as data . . . 24

Part III — services/: The Data Layer **26**

Naming conventions worth keeping . . . 26

Part III — The Read Economy **27**

The incident . . . 27

How Firestore bills . . . 27

The fix: CachedStreamPool . . . 27

Counting is not reading . . . 29

The same trap, one layer down: futures in build() . . . 29

The rules, as a checklist . . . 30

What optimisation cannot fix . . . 30

Reading the meter . . . 30

Part IV — Authentication and the Session **31**

The startup chain . . . 31
Email changes stay in sync . . . 31
Creating users without logging yourself out . . . 31
Deleting a user, and the orphan problem . . . 32

Part IV — CSV Import **33**

Two importers, one format . . . 33
The mirroring hazard . . . 33
Forgiving headers . . . 33
Two derived fields . . . 33
The duplicate-email trap . . . 34
The three phases of runImport . . . 34
Preview first, always . . . 34

Part IV — Hostels, Rooms and Allotment **35**

The model . . . 35
Why allotment is a transaction . . . 35
The seating policy . . . 36

Part IV — Academic Records: Batch, Year and Promotion **37**

The core idea . . . 37
The parser that will bite you . . . 37
Backfill versus promotion . . . 37
Graduation is an absence, not a delete . . . 38
Rooms reset every year . . . 38
The order that catches everyone . . . 38
Making promotion fast . . . 38
A validated worked example . . . 39
The counting trap that follows . . . 39

Part IV — Fines, Office Orders, Mess and the Rest **40**

Fines and office orders . . . 40
Mess fees . . . 40
Requests . . . 40
The audit log . . . 40

Part V — The Admin Tools **42**

Why they exist . . . 42
Setup . . . 42

The scripts . . . 42
What deletion actually clears . . . 42

Part V — Testing **44**

What is worth testing . . . 44
Why lib/utils/ is Firebase-free . . . 44

Part V — Deployment **45**

The command . . . 45
Hosting facts worth knowing . . . 45
The multi-site trap . . . 45
Deploying rules is separate, and non-negotiable . . . 45

Part VI — Every Trap This Project Hit **46**

Flutter and Dart . . . 46
Firestore . . . 46
Firebase Auth . . . 47
The domain . . . 47
Operational . . . 47

Part VI — Build It Again From Scratch **48**

1. Model the domain before writing any UI . . . 48
2. Design permissions before roles . . . 48
3. Set up Firebase and write the rules first . . . 48
4. Build the shell, the session and the registry . . . 48
5. One service per module, and never a query in a widget . . . 48
6. Feature by feature, three files at a time . . . 48
7. Write the tools before you need them . . . 48
8. Measure reads from the first day . . . 48
What I would do differently . . . 49

Appendix A — Command Reference **50**

Appendix B — The Permission Catalogue **51**

Appendix C — Glossary **52**

Closing **53**

How to Read This Manual

This document exists because you asked a fair question: *how much of my own project do I actually understand?*

The honest position is worth stating plainly. You supplied the domain knowledge — what a hostel office really does, how office orders are issued, that rooms reset every year, that a warden and a caretaker are different jobs. That knowledge is not recoverable from documentation; it came from you. The code was largely written in collaboration with an AI. Both of those things are true, and neither is diminished by the other. What has been missing is the layer in between: *why* each piece is shaped the way it is. That is what this manual restores.

What this covers, and what it deliberately does not

The project is roughly **45,000 lines**: 39,138 lines of Dart across 84 files, 599 lines of security rules, 1,922 lines of Node tooling, and 3,239 lines of tests. A literal line-by-line transcription would run past six hundred pages and would teach you *less*, not more, because the reasoning would drown in boilerplate. Ninety percent of a Flutter file is layout scaffolding you can read directly and understand on sight.

So this manual does something more useful. It covers:

- **Every subsystem**, end to end, with the code that matters quoted and explained
- **Every architectural decision**, with the alternative that was rejected and why
- **Every trap** the project hit, including the ones that shipped broken
- **Complete setup**, from an empty Firebase project to a deployed URL
- **The full data model**, collection by collection, field by field
- **The security rules**, explained as a system rather than a listing
- **A rebuild recipe** — how to construct this class of application from nothing

When you finish, you should be able to open any file in the repository and know why it is written that way. That is a stronger position than having memorised 45,000 lines.

How to use it

Read Parts I and II in order — they are the foundation, and the rest assumes them. After that, Parts III to V can be read in any order, or used as reference when you touch a subsystem. Part VI is the one to read twice: it is the accumulated scar tissue, and it is the part that transfers to your next project regardless of what you build it with.

Code excerpts are quoted verbatim from the repository. Where a file is named, it is a real path you can open.

Part I — The Problem and the Shape of the Answer

What the application is

Homi Hostel is a hostel management system for **Sant Longowal Institute of Engineering and Technology (SLIET)**. It runs the residential side of a campus of roughly 2,650 students across ten hostel blocks.

It answers a specific set of questions that a hostel office answers on paper:

- Who lives here, and in which room?
- Which beds are free, in which block, for which gender?
- Who has paid this month's mess fee?
- Who has been fined, for what, and has it been settled?
- What office orders have been issued, and against whom?
- What year is each student in — and what happens every July when they all move up one?

None of these are hard questions individually. The difficulty is that they are all the *same* data seen from different angles, and that the people asking them have different authority. A caretaker may move a student between rooms but must not be able to change their course. A student may see their own fine but nobody else's.

Who uses it

The user base is deliberately not a fixed list of roles. It is whatever roles the institution decides to create. The system ships with five starter templates — Chief Warden, Warden, Caretaker, BHS, Student — but these are *suggestions written once into the database*, not constants in the code. A Super Admin can invent "Block Supervisor" or "Mess Accountant" tomorrow and the entire application adapts without a code change.

This single decision shapes almost everything else, and it is covered properly in the chapter on the architectural rule.

The constraint that shaped the whole system

The project runs on Firebase's **Spark plan** — the free tier. This is not incidental. It is the single most influential technical constraint in the codebase, and it explains design choices that would otherwise look strange:

Limit	Value	Consequence
Document reads	50,000 / day	Reading the 2,650-student roster is 5% of the daily budget, once
Document writes	20,000 / day	A full promotion run costs ~2,600
Document deletes	20,000 / day	Clearing a roster is ~11,000
Cloud Functions	**not available**	No server-side code at all
Cloud Storage	**not available**	Images live as base64 inside documents

Every one of these shows up later as a design decision. No Cloud Functions is why deleting a user cannot remove their login. No Storage is why the college logo is a base64 string. The read budget is why there is a stream cache and why the dashboard counts instead of downloading.

A free tier is not merely "the paid thing with less of it". It removes entire architectural options. Design for the tier you are actually on, and write down which decisions exist only because of it — so that if you upgrade, you know what to revisit.

The shape of the answer

Three layers, and a strict rule about which may talk to which:

```
Widgets  (lib/screens/)      never touch Firestore
   |
   v
Services (lib/services/)     the only code that reads or writes
   |
   v
Firestore + Firebase Auth
   ^
   |
firestore.rules              the actual enforcement, independent of all the above
```

The critical thing about that diagram is the arrow on the right. `firestore.rules` is not a layer the app passes through — it is enforced by Google's servers on every request, whether that request came from your app, from `curl`, or from someone with a stolen ID token. The UI hiding a button is a convenience. The rules are the security.

Part I — Why Flutter and Firebase

The choice

Flutter for the client because one codebase targets web, Windows, Android and iOS, and the primary deployment is a web app that office staff open in Edge on a desktop. **Firebase** for the backend because the alternative — running a server, a database, and an auth system — is not something a single developer maintains for a college department for free.

What Flutter actually gives you here

Flutter renders its own widgets rather than wrapping platform controls. For this application that matters in one specific way: the dashboard is a dense, information-heavy layout with tables, charts, filter chips and drill-downs, and it looks and behaves identically on every target. There is no per-platform layout work.

The relevant pieces of Flutter used throughout:

- `StatelessWidget` / `StatefulWidget` — the two building blocks. Stateless when the widget is a pure function of its inputs; stateful when it must remember something across rebuilds.
- `StreamBuilder` — rebuilds a subtree whenever a stream emits. This is how live Firestore data reaches the screen.
- `FutureBuilder` — the one-shot equivalent, for data fetched once.
- `InheritedWidget` — propagates a value down the tree without passing it through every constructor. Used for the session.

What Firebase gives you

- **Firebase Auth** — accounts, sign-in, password management. Notably it authenticates against an *email address*; there is no username provider. This one fact forces the synthetic-email design covered later.
- **Cloud Firestore** — a document database. Collections contain documents; documents contain fields and may contain sub-collections. Queries are shallow by default and every query must be satisfiable by an index.
- **Firebase Hosting** — static hosting with automatic HTTPS, a global CDN, and free SSL.
- **Security Rules** — a declarative language, evaluated server-side per request, that decides whether a read or write is permitted.

What it cost

Being honest about the trade is more useful than selling the choice:

No server means no privileged operations. Firebase Auth's client SDK can only delete the *currently signed-in* account. So an admin cannot delete a student's login — the app deletes the profile document and the login survives as an orphan. This is worked around with Node scripts using the Admin SDK, run from a trusted machine. On the Blaze plan a Cloud Function would solve it properly.

No server means no secrets. Any logic in the client is visible to anyone who opens developer tools. This is why the rules layer exists and why it duplicates every check.

Firestore has no substring search. You cannot ask "students whose name contains 'kumar'". The workaround is to download the roster and filter in Dart — which is exactly why the user directory is the most expensive screen

in the app.

Documents are capped at 1 MB. With no Cloud Storage, images are base64 inside documents, and base64 inflates by about 33%. That gives a practical ceiling near 700 KB per image.

Part I — The One Architectural Rule

If you remember one thing from this manual, this is it.

One dashboard, permission-filtered

There is no separate student app and admin app. There is no `if (role == 'Warden')` anywhere in the codebase. A Student and a Super Admin run **the identical widget tree**. The only difference between what two people see is which navigation entries survive a permission filter.

The mechanism is `lib/core/nav_registry.dart`, a single constant list:

```
const List&lt;NavSection&gt; _allSections = [
  NavSection(
    title: 'People',
    items: [
      NavItem(
        id: 'users',
        label: 'User Management',
        icon: Icons.group_rounded,
        requires: [Perm.usersView],
        builder: _users,
      ),
      NavItem(
        id: 'academic',
        label: 'Academic Records',
        icon: Icons.event_available_rounded,
        requires: [Perm.academicView, Perm.academicManage],
        builder: _academic,
      ),
      ...
    ],
  ),
  ...
];
```

Each entry declares a `requires:` list. A function filters the list against the current session, and the sidebar renders whatever survives.

Adding a feature is therefore exactly three things: one entry in `_allSections`, one page file, one block in `firestore.rules`. Nothing else in the application needs to know the feature exists.

Why never branch on role names

```
// Never write this:
if (user.roleName == 'Warden') { ... }

// Always write this:
if (session.can(Perm.allotmentManage)) { ... }
```

Roles are **user-created data**, stored in `colleges/{id}/roles/`. The moment an administrator invents "Hostel Manager", every role-name check silently fails to match, and the feature disappears for a person who should have it — with no error, because nothing threw. A permission check asks the only question that actually matters: *is this person allowed to do this thing?*

The `excludes` escape hatch

A `NavItem` also has `excludes:`, which hides an entry from anyone holding those permissions:

```
NavItem(
  id: 'my-room',
  label: 'My Room',
  requires: [Perm.selfRoom],
```

```
  excludes: [Perm.allotmentManage],
  builder: _myRoom,
),
```

This exists for a real bug. `Session.can()` short-circuits to `true` for a Super Admin — they own the workspace, so they hold everything. That meant "My Room", a self-service page, appeared for the workspace owner, showing them a room they will never be allotted. `excludes` says: anyone who can *manage* allotment sees Room Allotment instead, which is the staff-facing view of the same information.

The permission catalogue

Permissions are plain strings of the form `module.action`, defined once in `lib/core/permissions.dart`:

```
class Perm {
  static const usersView = 'users.view';
  static const usersCreate = 'users.create';
  static const usersEdit = 'users.edit';
  static const usersDelete = 'users.delete';
  static const allotmentManage = 'allotment.manage';
  static const academicView = 'academic.view';
  static const academicManage = 'academic.manage';
  // ... 27 in total
}
```

They are strings rather than an enum for one reason: they are stored in Firestore documents and compared inside `firestore.rules`, which is a different language entirely. A Dart enum cannot cross that boundary; a string can, unchanged, and means the same thing on both sides.

Deciding staff from resident, without asking

One genuinely clever piece deserves attention. When creating a user, the form must decide whether to collect a full student record (guardian details, trade, batch, every CSV column) or a short staff record. It decides from permissions, not from the role name:

```
static bool isResident(Set&amp;lt;String&amp;gt; perms) =&amp;gt; !perms.any(
  (p) =&amp;gt;
      p.endsWith('.manage') ||
      p.endsWith('.viewAll') ||
      p.endsWith('.create') &amp;amp;&amp;amp; p != requestsCreate ||
      p == usersView ||
      p == usersEdit ||
      p == hostelsView,
);
```

Read the logic carefully. It does **not** whitelist student permissions. It asks: *does this role hold anything that reaches beyond itself?* Anything that manages a module, reads everyone's records, or opens the user directory is staff. A resident holds only self-service permissions — their own fines, their own requests, their own room.

Tested that way round deliberately. If it whitelisted student permissions, then a *new* self-service permission added next year would not be on the whitelist, and the role would be misfiled as staff. Testing for reach errs the safe way.

It is also why the check is not `managesHostels`. An accountant runs no hostel but is emphatically not a student, and should not be asked for a guardian's phone number.

Part II — Firebase From Zero

This chapter assumes nothing. Follow it and you will have a working backend.

Create the project

1. Go to `console.firebase.google.com` and click **Add project**.
2. Name it. The console derives a **project ID** — for this app, `homihostel-57391`. That ID is permanent and appears in every URL and config file.
3. Google Analytics is optional and unnecessary here.

Enable Authentication

1. **Build** → **Authentication** → **Get started**
2. Enable **Email/Password**. Do not enable "Email link (passwordless sign-in)" — it depends on Firebase Dynamic Links, which is being retired.
3. No other provider is needed. There is no Google or Facebook sign-in in this app.

Under **Authentication** → **Settings** → **Authorized domains**, your hosting domains are added automatically. Anything else that will serve the app must be listed here or sign-in will be rejected.

Create the Firestore database

1. **Build** → **Firestore Database** → **Create database**
2. Choose **Production mode** — this starts with everything denied, which is the correct default. Test mode allows the world to read and write your data for 30 days.
3. Choose a region close to your users (`asia-south1` for India). **This cannot be changed later.**

Wire the app to the project

Install the tooling and generate the config:

```
npm install -g firebase-tools
firebase login
dart pub global activate flutterfire_cli
flutterfire configure --project=homihostel-57391
```

`flutterfire configure` writes `lib/firebase_options.dart`, which holds one config block per platform:

```
static const FirebaseOptions web = FirebaseOptions(
  apiKey: 'AIza...',
  appId: '1:47219235653:web:fefa322011503d3116fc2c',
  messagingSenderId: '47219235653',
  projectId: 'homihostel-57391',
  authDomain: 'homihostel-57391.firebaseapp.com',
  storageBucket: 'homihostel-57391.firebasestorage.app',
);
```

The `apiKey` here is not a secret. It is a project identifier that ships inside every web bundle and is visible to anyone. It authorises nothing on its own — every request is still gated by Firebase Auth and by your security rules. Do not waste effort hiding it, and do not panic when you see it. The one true secret in this project is `tools/serviceAccountKey.json`, which bypasses all rules.

Note `authDomain`. Firebase Auth serves its own machinery from that domain — the password-reset and email-verification handlers live at `/__/auth/action` there. Disabling that Hosting site breaks password resets across the whole application.

Initialise Firebase in the app

`lib/main.dart` does four things before the first frame:

```
Future&lt;void&gt; main() async {
  WidgetsFlutterBinding.ensureInitialized();

  Object? startupError;
  try {
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
    );

    FirebaseFirestore.instance.settings = const Settings(
      persistenceEnabled: true,
      cacheSizeBytes: Settings.CACHE_SIZE_UNLIMITED,
    );

    if (kIsWeb) {
      await FirebaseAuth.instance.setPersistence(Persistence.SESSION);
    }
  } catch (e) {
    startupError = e;
  }
  ...
}
```

Each line earns its place:

The try/catch means a misconfigured `firebase_options.dart` produces a readable error screen rather than a blank white page. Debugging a blank page is miserable; the guard costs four lines.

`persistenceEnabled: true` turns on the local cache. Firestore then keeps a copy in IndexedDB and re-attaches listeners with a *resume token*, so a page reload asks "what changed since I last looked?" rather than paying for the whole roster again.

`CACHE_SIZE_UNLIMITED` matters more than it looks. The default cache is 40 MB — small enough that a 2,650-student roster can be evicted between visits and re-fetched in full, which is precisely the cost persistence was enabled to avoid.

`Persistence.SESSION` on web scopes the login to the browser tab. Firebase's web default is `LOCAL`, meaning the session survives closing the browser entirely — so opening the app on a shared office machine would land you in whoever signed in last. `SESSION` means closing the tab ends the session, while an accidental F5 mid-task still keeps you signed in. It is web-only in FlutterFire and throws on Windows and Android, hence the `kIsWeb` guard.

Deploy the rules

The rules file on your disk has **no effect whatsoever** until it is published:

```
firebase deploy --only firestore:rules
```

This project wraps that in `deploy-rules.bat`, which also records a hash of what was deployed, and `check-rules.bat`, which tells you whether the live rules match the file. That pairing exists because the gap between editing rules and deploying them is where confusing permission errors live.

Part II — The Data Model

The shape

```
colleges/{collegeId}
  |- roles/{roleId}              name, permissions[], isSystem
  |- hostels/{hostelId}          + denormalised roomCount/bedCount/occupiedBeds
  |    \- rooms/{roomNumber}     the room number IS the document id
  |- requests/{requestId}        leave / complaint / roomChange / other
  |- fines/{fineId}
  |- feeRecords/{recordId}       mess fees
  |- enrollments/{uid}_{session} one per student per academic year
  |- notices/{noticeId}
  |- officeOrders/{orderId}
  |- auditLog/{entryId}
  |- settings/config             one fixed document id
  \- mess/menu, mess/config      same single-document pattern

users/{uid}                    ROOT collection, deliberately not nested
```

Why `users` is a root collection

This is the first thing that looks wrong and is right.

Everything else lives under `colleges/{collegeId}`. Users do not. The reason is the login sequence: when someone signs in, Firebase Auth hands you a `uid` and **nothing else**. You do not know their college yet. If the profile lived at `colleges/{collegeId}/users/{uid}` you could not read it, because you do not know `collegeId` until you have read the profile. A root collection breaks the circle — `users/{uid}` is directly addressable from a uid alone.

The cost is that every query against `users` must carry an explicit `collegeId` filter, and the security rules must enforce that filter so one institution can never read another's roster. The rules do exactly that, per document, which is covered in the next chapter.

Deterministic document ids

Two collections use *computed* ids rather than random ones, and both do it for the same reason.

Rooms use the room number as the document id. `colleges/x/hostels/BH-5/rooms/204` *is* room 204. There is no lookup and no possibility of two documents claiming the same room.

Enrollments use `{uid}_{session}`:

```
static String idFor(String uid, String session) =&gt; '${uid}_$session';
```

This is what makes backfill and promotion **safe to re-run**. Writing the same student's 2026-27 record twice overwrites one document instead of creating two. A run that dies halfway through 2,600 students is recovered by running it again — there is no "did this already happen?" check to get wrong, because running it twice *is* the check.

> Deterministic ids are one of the highest-value habits in document databases. Any bulk operation you can express as `.set()` on a computed id becomes idempotent for free, and idempotency is what lets you recover from partial failure without writing a repair script.

Denormalisation, and where it is deliberate

Firestore has no joins. Rendering a list of 50 students with their room numbers must not cost 50 extra reads, so data is copied to where it is displayed.

Room and hostel are copied onto the user. `AppUser` carries `hostelId`, `hostelName`, `roomId`, `roomNumber`. The room document separately carries `occupantUids`. These two must never disagree, which is why allotment is a transaction.

Hostels carry counters. `roomCount`, `bedCount`, `occupiedBeds` are maintained incrementally rather than counted, so the hostels page renders occupancy without reading every room.

Enrollments duplicate almost everything — name, course, trade, batch, room. Partly for rendering, but mostly because they are *history*. Overwriting the room on a student profile every July would destroy which room someone held in a past session, which a warden eventually needs for a damage dispute or a deposit refund. The enrollment record is the permanent one.

Year belongs to the session, not the student

This is the single most important modelling decision in the application.

There is no `year` field on `AppUser`. A student's year lives on their **enrollment**, one per academic session:

```
/// `year` is stored here, per session — never on the student profile itself.
/// A student's Year 2 belongs to the 2026-27 session; asking "what year is
/// this student in" only makes sense once you also say "...in which
/// session", which is exactly what reading `enrollments/{uid}_{session}`
/// does and a `year` field on the profile never could.
```

If year lived on the profile, the annual promotion would be a destructive overwrite. You could never answer "who was in 3rd year in 2026-27" after the fact. Every year would erase the last.

The office order shape

An office order covers **one or more students**. One incident is one order: five students in the same fight get a single order naming all five, not five near-identical ones — which is how the office actually issues them, and it stores the scanned photograph once instead of five times.

Three fields must stay in step:

Field	Purpose
`students[]`	`{uid, name, regNo, fineId, fineAmount}` per person
`studentUids[]`	A flat array of the same uids
`fineTotal`	The sum of all amounts

`studentUids` looks redundant and is not. Firestore's `array-contains` **cannot look inside the maps of an array** — you cannot query `students[].uid`. Without the flat array, "orders against this student" is not expressible as a query at all.

`fineTotal` exists for a parallel reason on the rules side: security rules cannot iterate an array either, so a denormalised total is what tells the rules "this order levies money, therefore require `fines.manage`".

Amounts are per student and optional. The ringleader and the bystander do not pay the same, and a student can be named on an order with no fine at all. An order fining nobody is a warning or a suspension and writes no fine documents. Each fined student still gets their own `fines/{id}`, so each is settled, waived and counted independently.

Part II — The Security Rules, Explained

`firestore.rules` is 599 lines and is the only thing actually protecting the data. This chapter explains it as a system.

The governing idea

```
// Guiding principle: the client UI hides things, but these rules are what
// actually enforces them. Anyone can call the Firestore REST API directly,
// so every permission check in the app is repeated here.
```

Anyone holding a valid ID token can bypass your entire application and hit `firestore.googleapis.com` with `curl`. The rules are evaluated by Google's servers on every single request. They are not a second line of defence; they are the only line.

The helper functions

```
function signedIn() {
  return request.auth != null;
}

function me() {
  return get(/databases/$(database)/documents/users/$(request.auth.uid)).data;
}

function myCollege() {
  return me().collegeId;
}

function isSuperAdmin() {
  return signedIn() &amp;&amp; me().get('isSuperAdmin', false) == true;
}

function hasPerm(perm) {
  return signedIn() &amp;&amp; isActive() &amp;&amp; (
    isSuperAdmin() ||
    (
      me().get('roleId', null) != null &amp;&amp;
      perm in get(
        /databases/$(database)/documents/colleges/$(myCollege())/roles/$(me().roleId)
      ).data.get('permissions', [])
    )
  );
}
```

`hasPerm` is the workhorse, and it does exactly what the Dart `session.can()` does — reads the caller's role document and checks whether the permission string is in its list. The two implementations are deliberately parallel.

Two details worth noting. `me().get('isSuperAdmin', false)` uses the two-argument `get` with a default, so a document missing the field does not error. And `isActive()` is folded in, so **deactivating a user revokes every permission immediately**, without touching their role.

> `get()` inside rules costs a document read, billed to you. `hasPerm` performs up to two. This is a real cost on the free tier and a reason not to make rules more elaborate than they need to be.

Super Admin short-circuits everything

`hasPerm` returns true immediately for a Super Admin. This has a consequence that has cost this project debugging time:

Testing as Super Admin will not exercise the permission branches. Create a Warden and a Student to test properly. If you see "You don't have permission to do that" while signed in as Super Admin, your rules are out of date — redeploy them.

Tenant isolation, enforced per document

```
allow list: if signedIn()
            &amp;&amp; hasPerm('users.view')
            &amp;&amp; resource.data.collegeId == myCollege();
```

The critical subtlety: for a `list` operation, **rules are evaluated against each candidate document**. So this both requires the permission *and* makes it impossible to read another institution's users even if the client drops its `where` clause entirely. An attacker querying all of `users` with no filter gets nothing, because each document fails the check individually.

This is the difference between validating a query and validating results. Firestore does the latter, and it is much harder to get wrong.

Preventing privilege escalation

The most dangerous write in any permissions system is a user editing their own record. The rules restrict which *fields* may change:

```
function onlySelfEditableFields() {
  return request.resource.data.diff(resource.data).affectedKeys()
    .hasOnly(['name', 'phone', 'gender', 'email', 'updatedAt']);
}
```

`diff().affectedKeys().hasOnly([...])` computes which fields the write would change and requires that set to be a subset of the allowed list. Notably absent: `roleId`, `collegeId`, `isSuperAdmin`, `isActive`. **You cannot promote yourself.**

The same technique narrows delegated authority:

```
function onlyAllotmentFields() {
  return request.resource.data.diff(resource.data).affectedKeys()
    .hasOnly(['hostelId', 'hostelName', 'roomId', 'roomNumber',
              'roomCapacity', 'allottedAt', 'updatedAt']);
}
```

A warden holding `allotment.manage` can put a student in a room. They **cannot** rename that student, change their role, or reactivate a disabled account. Authority is scoped to the fields the job actually needs.

Promotion gets its own even narrower list, containing only the graduation flag — because academic staff marking someone graduated is not the same authority as filling their bed, and the reverse should not hold either.

And when an admin edits *someone else*:

```
resource.data.get('isSuperAdmin', false) == false &amp;&amp;
request.resource.data.get('isSuperAdmin', false) == false
```

Both the before and after state must be non-super-admin. You cannot create a Super Admin, and you cannot demote one.

Enrollments are append-only

```
// Never from the client — a wrong-year record is fixed by re-running
// promotion/backfill (both idempotent), not by deleting history.
allow delete: if false;
```

Academic history is not deletable from a browser under any circumstances. This is a deliberate integrity guarantee, and it has an operational consequence: clearing enrollments requires the Admin SDK, which bypasses rules. That is why `tools/delete-students.js` is the only thing that can do it.

The default deny

```
// Anything not matched above is denied.
match /{document=**} {
  allow read, write: if false;
}
```

The last block in the file. Any path not explicitly granted above is refused. New collections are locked until you write rules for them, which is the correct direction to fail.

Aggregation queries obey the same rules

`count()` and `sum()` are subject to the rules of the underlying query — but the `users` list rule constrains `resource.data`, which an aggregation never produces. Whether this worked at all was genuinely an open question when the dashboard was optimised.

It does work, and `rules-tests/` pins it along with the denials that matter: no `users.view`, no count; another college, no count; unscoped across all colleges, no count. If you touch that rule, run those tests before assuming the dashboard still loads.

Testing the rules

Rules are code and are tested like code, using the Firestore emulator:

```
cd rules-tests &amp;&amp; npm test
```

This boots `firebase emulators:exec --only firestore` against a throwaway project id, so it never touches live data. Write a test for every *denial* you care about, not just the permissions — a rule that accidentally allows everything passes all your "can they do it" tests.

Part III — The Codebase Layout

```
lib/core/          permissions, session, nav registry, theme, identity
lib/models/        AppUser, AppRole, Hostel/Room, HostelRequest, Mess,
                   Fine, Fee, Notice, OfficeOrder, AuditEntry, Enrollment,
                   CollegeSettings
lib/services/      one service per module; widgets never touch Firestore
lib/screens/       auth_gate, dashboard_shell, login
lib/screens/pages/ one file per page, plus *_view.dart for drill-downs
lib/utils/         pure functions, no Firebase, unit-testable
firestore.rules    security — the real enforcement
rules-tests/       emulator assertions against those rules
tools/             Node Admin-SDK scripts — import, delete, backup
test/              Dart unit and widget tests
```

The rule that keeps it navigable

Widgets never touch Firestore. Every read and write goes through a service. If you find `FirebaseFirestore.instance` inside a file under `screens/`, that is a bug, not a shortcut.

This is not architectural purity for its own sake. It buys three concrete things: every query is in one place when you need to optimise reads; a widget can be tested by handing it a plain list; and when a collection's shape changes, there is exactly one file to update.

Drill-downs are state, not routes

Pages hold drill-down state internally rather than pushing routes:

```
String? _openHostelId;
AppUser? _openUser;
```

Opening a hostel sets `_openHostelId` and the page renders the detail view instead of the list. It does not `Navigator.push`. The reason is the sidebar: pushing a route covers the whole screen including the navigation, so the user loses their place. Keeping it as state means the shell stays put and the back affordance is a button inside the content area.

Part III — core/: The Four Pillars

session.dart — who is looking at this screen

```
@immutable
class Session {
  final AppUser user;
  final AppRole? role;
  final String collegeName;
  final CollegeSettings settings;

  bool can(String permission) =&gt;
      user.isSuperAdmin || (role?.permissions.contains(permission) ?? false);

  bool canAny(List&lt;String&gt; permissions) =&gt; permissions.any(can);
}
```

Distributed through the tree by an `InheritedWidget`:

```
class SessionScope extends InheritedWidget {
  final Session session;

  @override
  bool updateShouldNotify(SessionScope oldWidget) =&gt;
      oldWidget.session.user.uid != session.user.uid ||
      oldWidget.session.role?.permissions != session.role?.permissions ||
      oldWidget.session.user.name != session.user.name ||
      oldWidget.session.collegeName != session.collegeName ||
      oldWidget.session.settings.updatedAt != session.settings.updatedAt;
}
```

`updateShouldNotify` decides whether dependents rebuild. It compares the fields that actually affect rendering rather than returning `true` blindly — returning `true` would rebuild every screen on every settings write.

Note `settings.updatedAt` as the settings comparison. Comparing whole settings objects would need deep equality across a dozen nested types; a server timestamp changes on every write and is one cheap comparison.

The dialog trap

This has already shipped broken once, so it gets its own warning.

```
static Session of(BuildContext context) {
  final scope = context.dependOnInheritedWidgetOfExactType&lt;SessionScope&gt;();
  if (scope == null) {
    throw FlutterError(
      'Session.of() found no SessionScope above this widget.\n'
      'If this is inside a showDialog/showModalBottomSheet builder, that is '
      'expected — the dialog is a separate route and cannot see the '
      'dashboard SessionScope. Read Session.of(context) in the calling '
      'page and pass the value into the dialog as a constructor argument.',
    );
  }
  return scope.session;
}
```

`showDialog` pushes onto the **root** Navigator. The dialog's context is therefore a *sibling* of the dashboard, not a descendant — and `SessionScope` sits below that Navigator. So `Session.of(context)` inside a dialog builder fails the moment the dialog opens: a full-screen red error, not a subtle bug.

`flutter analyze` cannot catch this. It compiles perfectly and fails at runtime. The fix is always the same — read the session in the calling page, pass what the dialog needs as constructor arguments. `BulkDeleteUsersDialog` and `RoomDetailDialog` both document the pattern, and `test/room_detail_dialog_test.dart` pins it by pumping the dialog with no scope above it.

The error message is long on purpose. A thrown error is a teaching opportunity, and the person hitting it at 2am is usually you.

identity.dart — logging in without an email

Firebase Auth has no username provider. It authenticates against an email address, full stop. Students do not have institutional email addresses. The bridge:

```
const String kSyntheticEmailDomain = 'homihostel.local';

class Identity {
  static bool looksLikeEmail(String input) =&gt; input.contains('@');

  static String toAuthEmail(String input) {
    final v = input.trim().toLowerCase();
    if (v.isEmpty) return v;
    return looksLikeEmail(v) ? v : '$v@$kSyntheticEmailDomain';
  }

  static bool isSynthetic(String email) =&gt;
      email.toLowerCase().endsWith('@$kSyntheticEmailDomain');

  static String display(String email) =&gt;
      isSynthetic(email) ? email.split('@').first : email;
}
```

A student types `2110910`; Firebase sees `2110910@homihostel.local`. Staff type a real address and it passes through untouched. The domain is non-routable on purpose — nothing should ever attempt to deliver mail there.

Never show a raw synthetic address in the UI. Always `Identity.display()`. A student seeing `2110910@homihostel.local` on their profile will reasonably conclude the system has their email wrong.

The consequence, stated plainly

A synthetic address has no inbox, so **Firebase's password-reset email can never reach a student**. Resetting a student's password is an admin action, not a self-service one. Anyone signing in with a real address keeps working resets.

The security weakness you should know about

```
static String derivedPassword(String registrationNumber) {
  final v = registrationNumber.trim();
  return v.length &gt;= 6 ? v : v.padRight(6, '0');
}
```

The starting password **is the registration number**, padded to Firebase's six-character minimum. Username and password are the same string.

This exists because bulk-provisioning 2,649 accounts requires a deterministic password — there is no inbox to send a generated one to, and no practical way to distribute 2,649 individual secrets through the app. It is a real trade, and the importer prints "tell them to change it from My Profile after first login".

But hoping thousands of students each change a password is not a control. Registration numbers are printed on ID cards and are sequential. Until a student changes it, anyone who knows their roll number can sign in as them. If you rebuild this, consider forcing a password change on first login, or seeding from roll number plus date of birth and distributing on paper.

theme.dart

Centralises colour and typography so a college can rebrand from System Settings. `AppColors.apply(accent:, dark:)` recomputes the palette at runtime. The login screen reads a locally cached copy (`ThemeCache`) so it

matches the workspace before Firestore has loaded — otherwise every visit flashes the default blue before repainting.

Part III — models/: The Data Shapes

Every model follows the same three-part contract.

```
class Enrollment {
  final String id;
  final String uid;
  final String session;
  final int year;

  factory Enrollment.fromMap(String id, Map&lt;String, dynamic&gt; m) =&gt; Enrollment(
    id: id,
    uid: m['uid'] as String? ?? '',
    session: m['session'] as String? ?? '',
    year: (m['year'] as num?)?.toInt() ?? 1,
  );

  Map&lt;String, dynamic&gt; toMap() =&gt; {
    'uid': uid,
    'session': session,
    'year': year,
  };
}
```

1. **Immutable fields.** No setters. A changed model is a new object.
2. `fromMap`**, total and defensive.** Every field uses `as X?` with a fallback. A document missing a field, or holding the wrong type, produces a usable object rather than a crash. This matters because documents written by an older version of the app are still out there.
3. `toMap`, the inverse.

Note `(m['year'] as num?)?.toInt()`. Firestore returns numbers as `int` or `double` depending on how they were written; casting to `num` first accepts both.

Computed properties belong on the model

```
bool get isAllotted =&gt; roomId != null &amp;&amp; hostelId != null;
bool get isGraduated =&gt; status == 'graduated';
```

`isAllotted` requiring **both** ids is load-bearing. A student with a hostel but no room is *not* allotted — they have a block preference, not a bed. Getting this wrong would make thousands of students look housed when no bed was assigned. Because it is one getter, every screen agrees.

CollegeSettings — configuration as data

Trades, fine categories and course rules are **not constants in the code**. They are editable settings:

```
class CourseRule {
  final String course;
  final int totalYears;
  final CourseSeating seating;
  final int sharedCapacity;
}

const List&lt;CourseRule&gt; kDefaultCourseRules = [
  CourseRule(course: 'UG',  totalYears: 4, seating: CourseSeating.meritSingle),
  CourseRule(course: 'ICD', totalYears: 3, seating: CourseSeating.alwaysShared),
  CourseRule(course: 'PG',  totalYears: 2, seating: CourseSeating.alwaysSingle),
];
```

The constants are **seeds**, written once at registration, editable afterwards. A different college has different courses, and hard-coding SLIET's would make the app single-tenant in practice.

Reads fall back to the seed when the list is empty, so a workspace that never opened System Settings behaves identically to one that did:

```
List&lt;CourseRule&gt; get _rules =&gt;
    courseRules.isEmpty ? kDefaultCourseRules : courseRules;

int totalYearsFor(String? course) =&gt; _ruleFor(course)?.totalYears ?? 4;
```

Part III — services/: The Data Layer

Every service is a lazy singleton:

```
class EnrollmentService {
  EnrollmentService._();
  static final EnrollmentService instance = EnrollmentService._();

  final FirebaseFirestore _db = FirebaseFirestore.instance;

  CollectionReference&lt;Map&lt;String, dynamic&gt;&gt; _enrollments(String collegeId) =&gt;
      _db.collection('colleges').doc(collegeId).collection('enrollments');
}
```

A private constructor plus a static `instance` gives one object per service for the process lifetime — which is what allows a service to hold a stream cache that outlives any individual screen.

The `_enrollments(collegeId)` helper means the collection path is written once. Every method builds from it, so a path change is one edit.

Naming conventions worth keeping

Prefix	Returns	Meaning
`watchX`	Stream	Live, pooled, updates as data changes
`getX`	Future	One-shot read
`countX`	Future of int	Aggregation — no documents transferred
`deleteForStudent`	Future of int	Bulk cleanup, returns how many went

Consistency here is what lets you predict the cost of a call from its name, which on a metered database is not a small thing.

Part III — The Read Economy

This chapter is the one that will save you money on any Firebase project.

The incident

A 2,500-student roster once burned the entire 50,000-read daily budget in about nineteen screen opens. Six different pages each called `watchUsers()` from inside `build()`, each one paying for the whole collection — and because the calls sat inside `build()`, a filter change or even a keystroke in a search box could pay for it again.

How Firestore bills

- **You pay per document delivered.** A query returning 2,649 documents costs 2,649 reads.
- **A listener bills for its initial snapshot**, then only for documents that change.
- `snapshots()` **returns a brand-new listener every time it is called.**
- **Aggregations** (`count()`, `sum()`) bill one read per 1,000 index entries scanned, minimum one, and transfer no document data.

The third point is the trap. `StreamBuilder` compares the stream it was given against the previous one; if the object differs, it tears down the old subscription and opens a new one — paying for the whole query again. Building a stream inside `build()` therefore re-reads the collection on **every rebuild**.

The fix: CachedStreamPool

`lib/services/stream_cache.dart` solves both halves.

```
class CachedStream&lt;T&gt; extends Stream&lt;T&gt; {
  final Stream&lt;T&gt; Function() _open;

  StreamSubscription&lt;T&gt;? _upstream;
  final StreamController&lt;T&gt; _out = StreamController&lt;T&gt;.broadcast();

  T? _latest;
  bool _hasLatest = false;

  void _connect() {
    if (_upstream != null || _out.isClosed) return;
    _upstream = _open().listen((value) {
      _latest = value;
      _hasLatest = true;
      if (!_out.isClosed) _out.add(value);
    }, onError: (Object e, StackTrace st) {
      _error = e;
      _hasLatest = false;
      if (!_out.isClosed) _out.addError(e, st);
    });
  }
}
```

One upstream subscription, opened on the first listener and held for the rest of the session. The second screen and the fiftieth attach to the same live query and cost nothing.

A stable stream object, handed out by a pool keyed on the query parameters:

```
class CachedStreamPool&lt;T&gt; {
  final Map&lt;String, CachedStream&lt;T&gt;&gt; _byKey = {};

  Stream&lt;T&gt; stream(String key, Stream&lt;T&gt; Function() open) =&gt;
      _byKey.putIfAbsent(key, () =&gt; CachedStream&lt;T&gt;(open));
}
```

`putIfAbsent` returns the *same instance* for the same key, so `StreamBuilder` sees an unchanged stream across rebuilds and never re-subscribes. That is what stops a keystroke from re-reading the roster.

The replay subtlety

A page opened after the data arrived must not sit on a spinner. So late subscribers are replayed the latest value — and the way that is done matters:

```
Stream&lt;T&gt; _replayed() =&gt; Stream&lt;T&gt;.multi((controller) {
  final sub = _out.stream.listen(
    controller.add,
    onError: controller.addError,
    onDone: controller.close,
  );
  controller.onCancel = sub.cancel;

  final err = _error;
  if (err != null) {
    controller.addError(err, _trace);
  } else if (_hasLatest) {
    controller.add(_latest as T);
  }
});
```

`Stream.multi` rather than `async*`, because the order of the two steps matters and only this version gets it right. The obvious implementation — yield the cached value, then `yield*` the controller — reads the cache and subscribes in **separate microtasks**. A snapshot landing in the gap is added to a broadcast controller with no listener yet and is dropped silently. The subscriber keeps the value it replayed and never learns it went stale.

Here the subscribe happens first and the replay second, inside one synchronous callback. Nothing can arrive in the gap and nothing can arrive out of order.

> This is the kind of bug that never reproduces on your machine and corrupts one screen for one user occasionally. If you write a caching layer, write this test.

Errors are cached too

`_error` is retained so a page opened after a permission failure shows the error rather than an eternal spinner. A cache that only remembers successes turns every failure into a hang.

The upstream is never cancelled

Deliberately. Navigating away from a page and back is precisely the case worth making free; dropping the query there would re-read everything on return. The cost is one live listener, which Firestore only bills for documents that actually change.

Which means it must be torn down at sign-out

```
static Future&lt;void&gt; disposeAll() async {
  for (final s in [..._live]) {
    await s.dispose();
  }
  _live.clear();
  for (final p in _pools) {
    p._byKey.clear();
  }
}
```

Called from `AuthService.signOut()` **before** `_auth.signOut()`. Miss this and the next person to log in inherits the previous user's data and a listener they have no permission to hold — which surfaces as a permission error on a screen that never asked for anything.

Note that the pools are cleared too. Without that, the next caller is handed a disposed stream that can never reconnect, because `_connect()` refuses to run once `_out.isClosed`.

Counting is not reading

The dashboard wants seven integers. Deriving them from the roster cost one billed read per student — 2,650 every time the app opened, before anyone clicked anything.

```
Future&lt;int&gt; countUsers(
  String collegeId, {
  String? roleId,
  bool? isActive,
  bool? hasNoRole,
}) async {
  var q = _usersIn(collegeId);
  if (roleId != null) q = q.where('roleId', isEqualTo: roleId);
  if (hasNoRole == true) q = q.where('roleId', isNull: true);
  if (isActive != null) q = q.where('isActive', isEqualTo: isActive);

  final snap = await q.count().get();
  return snap.count ?? 0;
}
```

`count()` answers from the index. The same figure costs about **3 reads instead of 2,650**, and no document data crosses the network.

Counting per role is what makes it cheap and complete at once: the role documents are already streamed (six of them), and one count each yields the total, the resident/staff split and the breakdown chart.

The trade

A `count()` is a Future, not a Stream — the dashboard shows a snapshot from when it loaded. For a summary screen that is right. For Room Allotment, where two wardens genuinely race for the last bed, it is wrong, and that screen still streams.

Memoise, or it defeats itself

```
Future&lt;UserCounts&gt;? _counts;
String? _countsKey;

Future&lt;UserCounts&gt; _countsFor(List&lt;AppRole&gt;? roles) {
  if (roles == null) return Future.value(const UserCounts());
  final key = roles.map((r) =&gt; r.id).join(',');
  if (_countsKey != key || _counts == null) {
    _countsKey = key;
    _counts = DataService.instance.countUsersByRole(widget.collegeId, roles);
  }
  return _counts!;
}
```

Without this, every rebuild of the dashboard — a fee arriving, a notice landing — re-fires the whole batch of count queries, which is precisely the cost the change exists to remove.

The same trap, one layer down: futures in build()

There is no pool for Futures, which makes this harder to spot:

```
// WRONG — a fresh Firestore read on every single rebuild
FutureBuilder&lt;Room?&gt;(
  future: AllotmentService.instance.findRoom(...),
)
```

`MyRoomPage` was a `StatelessWidget` doing exactly this, twice, on the page every resident opens. The fix is to memoise in a `State` field, keyed on whatever genuinely invalidates it:

```
Future&lt;Room?&gt;? _room;
String? _roomKey;

Future&lt;Room?&gt; _roomFor(AppUser user) {
```

```
  final key = '${user.collegeId}/${user.hostelId}/${user.roomId}';
  if (_roomKey != key || _room == null) {
    _roomKey = key;
    _room = AllotmentService.instance.findRoom(...);
  }
  return _room!;
}
```

Keyed on the allotment, so a student who is actually moved still refetches, while an ordinary rebuild does not.

The rules, as a checklist

1. Never call `.snapshots()` directly from a widget.
2. Never build a new stream object per rebuild — go through a pool.
3. Never build a Future inside `build()` — memoise it in State, keyed.
4. Prefer `count()` / `sum()` when you only need a number.
5. Memoise aggregations, or they re-fire on every rebuild.
6. Put a `.limit()` on anything unbounded that will grow.
7. `StreamCaches.disposeAll()` on sign-out, before the auth state changes.

What optimisation cannot fix

Bulk operations. A full wipe-and-reimport cycle costs roughly 11,000 writes and 11,000 deletes regardless of how well the read path is tuned. Two such cycles in a day will exhaust the free tier, and no amount of stream pooling changes that. Plan test cycles accordingly.

Reading the meter

Firebase Console, then Firestore, then Usage shows reads, writes and deletes per hour. When something feels slow, check there first — on Spark, "slow" and "quota exhausted" look identical from inside the app, because a failed aggregation leaves a `FutureBuilder` with neither data nor a handled error.

Quotas reset at **midnight Pacific Time**, which is about 12:30 PM IST.

Part IV — Authentication and the Session

The startup chain

`AuthGate` is the root widget below `MaterialApp`. It listens to `authStateChanges` and decides what the whole application shows:

```
authStateChanges
   |
   +-- null  ------------------&gt; LoginScreen
   |
   \-- User  --&gt; watchProfile(uid)     users/{uid}
                  |
                  +-- null ---------&gt; "profile missing" screen
                  |
                  \-- AppUser --&gt; watchRole(collegeId, roleId)
                                     |
                                     \--&gt; watchCollegeName(collegeId)
                                            |
                                            \--&gt; SettingsService.watch()
                                                   |
                                                   \--&gt; SessionScope
                                                          |
                                                          \--&gt; DashboardShell
```

Four nested `StreamBuilder`s, each adding one piece to the `Session`. Because every one is a live stream, a permission change made by an admin reaches the affected user's sidebar **without them re-logging in**. Revoking access is immediate.

`authStateChanges` rather than a one-shot check is deliberate: sign-out anywhere in the app simply works, with no manual navigation and therefore no chance of a stale route stack.

Email changes stay in sync

```
await user.verifyBeforeUpdateEmail(newEmail);
```

`verifyBeforeUpdateEmail` only changes the Auth email once the confirmation link is clicked, so there is no destructive window where Auth and Firestore disagree. On the next sign-in, `_SessionLoader._syncEmail` notices `authUser.email != profile.email` and writes the new address into the profile — skipped for synthetic student addresses, which must never be overwritten.

Creating users without logging yourself out

`createUserWithEmailAndPassword` signs the **new** user in on the default `FirebaseAuth` instance — which would log the admin out mid-task. The workaround:

```
// Spin up an isolated FirebaseApp for provisioning, then throw it away.
final temp = await Firebase.initializeApp(
  name: 'provisioner-$timestamp',
  options: DefaultFirebaseOptions.currentPlatform,
);
final auth = FirebaseAuth.instanceFor(app: temp);
await auth.createUserWithEmailAndPassword(email: ..., password: ...);
```

A secondary `FirebaseApp` has its own auth instance. The new account is created there, the admin's session on the default instance is untouched, and the temporary app is disposed afterwards.

> This is the right pragmatic call with no backend. The production-grade answer is a Cloud Function using the Admin SDK — which also solves account deletion. Move to it when you can afford Blaze.

Deleting a user, and the orphan problem

`DataService.deleteUserCompletely` unwinds everything attached to a person:

1. Attempt to delete the Firebase Auth account
2. Vacate their room (frees the bed, fixes the counter)
3. Delete their fines, requests, mess-fee records
4. Delete the profile document
5. Write an audit entry

Steps 2 to 4 are each wrapped in their own try/catch. A fines cleanup failure must not block removing the person, and whoever can delete a user may not hold `fines.manage`.

Step 1 is the one that fails. The client SDK can only delete the **currently signed-in** account, so the trick is to sign in as the target on an isolated app and call `.delete()`:

```
final candidates = &lt;String&gt;{
  if ((user.enrollmentNo ?? '').trim().isNotEmpty)
    Identity.derivedPassword(user.enrollmentNo!.trim()),
  if (Identity.isSynthetic(user.email))
    Identity.derivedPassword(Identity.display(user.email)),
}.toList();
```

Only the **derived** password is ever tried — the app has no store of real ones and must not pretend otherwise. If the student changed their password, or it is a staff account with a custom one, every candidate fails and the Auth account survives.

The profile is then deleted **unconditionally**. That is a deliberate trade: a failed Auth deletion must not leave the person un-removable. But it creates an **orphan** — a login with no profile behind it.

The symptom appears much later: "email already in use" when re-adding that person. The fix is Admin-SDK only:

```
node delete-students.js --orphans --commit
```

Part IV — CSV Import

Bulk-loading 2,649 students is the heaviest operation in the application, and it exists in two implementations that must agree exactly.

Two importers, one format

	In-app importer	`tools/import-students.js`
Runs in	The browser	Your machine
Credentials	The signed-in admin	Service-account key
Account creation	Public sign-up endpoint	Admin SDK
Throttled	Yes, aggressively	No
Good for	5 students mid-semester	2,649 at session start

Firebase treats a browser as untrusted and rate-limits account creation after a dozen or so, which is why a 30-row import crawls and a 2,649-row one belongs in the script.

The mirroring hazard

`tools/lib.js` deliberately mirrors Dart code and **must stay in step**. CSV parsing, header aliases, synthetic login emails, semester and batch derivation, and the trade list all exist twice:

```
tools/lib.js   &lt;-&gt;   lib/services/csv_import.dart
                      lib/core/identity.dart
                      lib/models/app_user.dart
```

A student imported by the script and one imported in-app must come out **byte-identical**, or the fines dashboard buckets them separately. The mapping table lives at the bottom of `tools/README.md`.

> Duplicated logic across languages is a genuine design smell, accepted here because the alternative — a shared backend — is not available on Spark. If you take one thing from this: when you must duplicate, write down *where* the duplicate is, next to both copies.

Forgiving headers

```
const HEADER_ALIASES = {
  'student name': 'name', 'name': 'name',
  'registration number': 'registrationNo',
  'enrollment': 'registrationNo', 'enrollmentno': 'registrationNo',
  'branch': 'trade', 'trade': 'trade',
  'semester': 'sem', 'sem': 'sem',
  'hostel number': 'hostel', 'room no.': 'room',
  ...
};
```

Real spreadsheets come from a dozen offices with a dozen conventions. Normalising headers costs one lookup table and removes an entire class of support question.

Only `name` plus either `registrationNo` or `email` are required. Everything else is optional.

Two derived fields

Batch from the registration number. The two-digit admission year leads the number:

```
/** "2110910" -&gt; "2021-22" */
```

```
export function batchFromRegistrationNo(regNo) {
  const v = (regNo ?? '').trim();
  if (v.length &lt; 2) return null;
  const yy = parseInt(v.slice(0, 2), 10);
  if (Number.isNaN(yy)) return null;
  const start = 2000 + yy;
  if (start &lt; 2000 || start &gt; new Date().getFullYear() + 1) return null;
  return `${start}-${String((start + 1) % 100).padStart(2, '0')}`;
}
```

The range check matters. Without it a malformed number silently produces a batch of "1987-88" and that student's year is wrong forever.

Semester accepts `5`, `Sem 5`, `5th` or the Roman `V`.

The duplicate-email trap

A row with an `email` value logs in with *that* address. Siblings sharing one parent's address therefore share a login — and since the importer matches existing students by that same key, the second row is read as an **update of the first and silently overwrites them**.

One student vanishes and nothing reports an error. Hence a preflight tool:

```
node fix-duplicate-emails.js ../finalsheet.csv
```

It writes `finalsheet.fixed.csv` and never touches the input. Later rows of each colliding group get a synthetic `{registrationNo}@homihostel.local` address plus a `notes` cell recording what they shared and with whom. It exits without writing if two rows carry the same registration number, which no generated login can separate.

The three phases of runImport

This is the most carefully tuned code in the project.

```
Phase 1  Updates          plain Firestore writes, 10 concurrent
Phase 2  Account creation runPooled lanes, narrowing to 1 on throttle
Phase 3  Allotment        strictly sequential
```

Phase 2 is deliberately timid. Account creation runs on `runPooled` lanes that **narrow to one permanently** the first time Firebase answers `too-many-requests`:

> Never "tune up" the create concurrency to match the Node script's. That script is on a different endpoint with a service-account key, and the block this one risks tripping is **project-wide** — it affects real student sign-ins, not just your import.

The pool logic is tested in `test/run_pooled_test.dart` against a fake task, so the narrowing behaviour is verified without touching Firebase.

Phase 3 is sequential because rows can name the same room and the room cache is shared mutable state. Two rows racing for the last bed in room 204 must not both win.

Preview first, always

```
node import-students.js finalsheet.csv          # dry run
node import-students.js finalsheet.csv --commit # apply
```

Without `--commit` nothing is written and you get a `NEW` / `UPDATE` / `SKIP` line per row with a reason. **Re-running is safe** — a registration number that already exists is updated, never duplicated — so a run that dies halfway is recovered by running it again.

Part IV — Hostels, Rooms and Allotment

The model

A hostel has a gender policy, a set of rooms, and denormalised counters. A room's document id **is** its number, and it carries `capacity`, `occupantUids`, `status` and optional `features` and `rentPerBed`.

Occupancy is stored in two places at once: `occupantUids` on the room, and `hostelId`/`roomId`/`roomNumber` on the user. They must never disagree.

Why allotment is a transaction

> Two wardens clicking the last free bed simultaneously would both read "1 free" and both write.

A batch cannot prevent this — batches are atomic writes, not atomic read-modify-write. Only a transaction re-reads and re-validates at commit time.

```
await _db.runTransaction((tx) async {
  // --- all reads first; Firestore forbids a read after a write ---
  final roomSnap = await tx.get(roomRef);
  final userSnap = await tx.get(userRef);
  final settingsSnap = await tx.get(settingsRef);

  final fresh = Room.fromMap(roomSnap.id, roomSnap.data()!);
  final freshUser = AppUser.fromMap(userSnap.id, userSnap.data()!);

  // Re-check against what is actually in the database right now, not what
  // the warden's screen was showing when they clicked.
  if (freshUser.isAllotted) {
    throw AllotmentFailure(
      '${freshUser.name} was just allotted ${freshUser.roomLabel} by someone else.',
    );
  }
  if (fresh.status != RoomStatus.active) {
    throw AllotmentFailure(
      'Room ${fresh.number} is marked ${fresh.status.label} and cannot take students.',
    );
  }
  if (fresh.occupied &gt;= fresh.capacity) {
    throw AllotmentFailure(
      'Room ${fresh.number} filled up while you were choosing '
      '(${fresh.occupied}/${fresh.capacity}).',
    );
  }

  tx.update(roomRef, {
    'occupantUids': [...fresh.occupantUids, student.uid],
  });
  tx.update(userRef, {
    'hostelId': hostel.id,
    'hostelName': hostel.name,
    'roomId': fresh.id,
    'roomNumber': fresh.number,
    'allottedAt': FieldValue.serverTimestamp(),
  });
  tx.update(hostelRef, {'occupiedBeds': FieldValue.increment(1)});
});
```

Four rules are visible here and all four are worth internalising:

All reads before any write. Firestore forbids a read after a write inside a transaction. Violating it throws at runtime, not at compile time.

Re-validate everything inside the transaction. The checks before the transaction are for a fast, friendly error. The checks *inside* are the ones that are actually true, because Firestore retries the whole closure if a read document changed underneath.

Error messages name the cause. "Room 204 filled up while you were choosing (3/3)" tells a warden exactly what happened. "Allotment failed" starts a support conversation.

`FieldValue.increment(1)` is atomic server-side. Reading the counter and writing back would lose updates under concurrency; increment cannot.

The seating policy

Room entitlement is derived, not stored:

```
int? requiredRoomCapacity({
  required CourseRule? rule,
  required int year,
  required bool singleRoomEligible,
}) {
  if (rule == null) return null;
  return switch (rule.seating) {
    CourseSeating.alwaysShared =&gt; rule.sharedCapacity,
    CourseSeating.alwaysSingle =&gt; 1,
    CourseSeating.meritSingle =&gt;
      year &lt;= 2
          ? rule.sharedCapacity
          : (singleRoomEligible ? 1 : rule.sharedCapacity),
  };
}
```

A `null` rule means **no enforcement**, not "shared by default". A college running a course nobody has configured should leave room type to staff judgement rather than have the app silently guess wrong.

`singleRoomEligible` is a permanent flag — once earned it stays true, so a student who earned a single in year 3 still has one in year 4.

Part IV — Academic Records: Batch, Year and Promotion

This is the most conceptually interesting subsystem in the application.

The core idea

Year is not a property of a student. It is a property of a **student in a session**.

```
int? yearFromBatch(String? batch, String? currentSession) {
  final batchStart = startYearOf(batch);
  final sessionStart = startYearOf(currentSession);
  if (batchStart == null || sessionStart == null) return null;
  return sessionStart - batchStart + 1;
}
```

A batch says which session the student's Year 1 was. From that and the session running now, the year follows by subtraction — no course-specific branch, and no special case for lateral entry, because a lateral entrant carries the batch of the cohort they were slotted into.

The parser that will bite you

```
int? startYearOf(String? span) {
  final v = span?.trim() ?? '';
  final dash = v.indexOf('-');
  if (dash &lt;= 0) return null;
  return int.tryParse(v.substring(0, dash));
}
```

Every session and batch is a **span with a dash**: `2026-27`, not `2026`. A bare year returns `null`, which makes `yearFromBatch` return `null` **for every student**, which makes backfill write nothing and report all 2,649 as needing a batch fix.

This happened. The error message blamed 2,649 innocent batch fields instead of the one malformed settings field. If you rebuild this, **validate the session format at the point of entry**.

Backfill versus promotion

Two operations that look similar and are opposites.

	Backfill	Promotion
Input	`batch` + current session	The **previous enrollment's** year
Formula	`sessionStart - batchStart + 1`	`previous.year + 1`
Graduation check	**None**	Yes
When	Once, ever, to bootstrap	Every July
Re-runnable	Yes	Yes

Promotion deliberately never consults batch math. A repeater's actual year can disagree with what their batch would predict, and promotion must move students from where they *are*, not recompute where they "should" be. Off-track students are *reported*, never blocked:

```
final batchYear = yearFromBatch(previous.batch, toSession);
final offTrack = batchYear != null &amp;&amp; batchYear != newYear;
```

Backfill has no graduation ceiling. It writes whatever the arithmetic gives, even year 6 of a 4-year course. That is correct for a bootstrap — you are describing history — but it means backfilling into too late a session produces impossible years.

Graduation is an absence, not a delete

```
if (hasGraduated(newYear, totalYears)) {
  if (user.isAllotted) {
    await AllotmentService.instance.vacate(collegeId: collegeId, student: user);
  }
  await _db.collection('users').doc(user.uid).update({
    'status': 'graduated',
    'updatedAt': FieldValue.serverTimestamp(),
  });
  return PromotionResult.graduated(user, previous.year);
}
```

No next-session enrollment is written. The student's profile is flagged, their room is freed, and they drop out of every session-scoped query — **without a single document being removed**. Their entire history remains queryable years later.

Rooms reset every year

Except a Single already earned by merit:

```
final keepsRoom = previous.roomCapacity == 1;
```

A student being reset is **actually vacated** through `AllotmentService`, not merely cleared on the enrollment record. Otherwise the physical room stays phantom-occupied by someone whose new-session record shows no room at all.

The order that catches everyone

Promote first. Change the session setting second.

Promotion reads the *current* setting as the session to promote **from**. Switching System Settings to the new session first is the natural thing to try, and it leaves you with an empty roster and a disabled button. The page spells this out rather than leaving a silent dead end.

```
2025-26:  set session -&gt; BACKFILL -&gt; (year runs)
             |
   July:  PROMOTE to 2026-27   &lt;-- while settings still say 2025-26
             |
2026-27:  flip settings to 2026-27
```

Making promotion fast

The original implementation called `promoteOne` in a loop — two or three sequential network round-trips per student. Across 2,600 students that is thousands of round-trips one after another, and a run took minutes.

The rewrite:

1. **Read once.** Take the `fromSession` roster the page is already streaming, or fetch it in one query. Zero extra reads instead of one `get()` per student.
2. **Decide in memory.** A shared `_planFor` computes every outcome with no I/O, so `promoteOne` and `promoteAll` cannot drift apart.
3. **Vacate only the few** who actually hold a room — 60 of 2,649 in practice, not all of them.
4. **Batch the rest**, 450 writes at a time.

Thousands of round-trips became about six. The lesson generalises: **separate the decision from the write**, and the batching becomes obvious.

A validated worked example

Starting from a real 2,649-student roster in session 2025-26, with UG at 4 years, ICD at 3 and PG at 2:

Promotion into	Graduate	Left enrolled
2026-27	765	1,884
2027-28	686	1,198
2028-29	835	363
2029-30	363	**0**

765 + 686 + 835 + 363 = 2,649. Every student accounted for, the roster drains to zero with no further imports, and the last cohort leaves exactly when its four years are up. This was run end to end against live data and every figure matched the prediction.

The counting trap that follows

`isGraduated` is checked in exactly **one** screen. Everywhere else — Room Allotment, Student Overview, User Management, the dashboard counts — graduates are still treated as current students. Since graduation vacates their room, they pile into the allotment queue as students "awaiting a bed".

If you rebuild this: decide early whether "graduated" means *hidden* or *filterable*, and apply that decision everywhere at once. A status field that only one screen respects is worse than no status field, because it looks like it works.

Part IV — Fines, Office Orders, Mess and the Rest

Fines and office orders

An office order is the paper artefact; fines are its financial consequence. `FineService.imposeWithOfficeOrder` writes both in **one batch**, so an order can never exist without its fines or vice versa.

Each fined student gets their own `fines/{id}` document, so each is settled, waived and counted independently — while the scanned photograph is stored once on the order.

The known cost problem

Office-order photos are base64 **inside** the order documents, and the register streams every one. Forty orders is roughly **35 MB** transferred to render forty rows of text. The reads are cheap — forty documents — but the bandwidth and memory are not, and it worsens with every order issued.

The fix is to move the image to `officeOrders/{id}/media/photo`, letting the register stream metadata only and fetch a photo when someone actually opens one. Not done. Know about it before adding anything else to that page.

Mess fees

The app **never takes money**. It records payment *status*, which is a much smaller and safer problem than handling payments.

Records live at `feeRecords/{period}_{uid}` — deterministic again, so marking someone paid twice updates one row rather than creating a duplicate. That matters when two clerks work through the same list.

The period is a sortable `YYYY-MM` string rather than a Timestamp, which is what allows a single-field range query (`period >= '2026-01'`) with **no composite index**.

The dashboard's revenue trend uses `sum()` aggregations rather than streaming records. A college of 2,500 students paying monthly puts ~15,000 documents behind a seven-month window — six times the roster, and 30% of the daily read budget on one screen.

Requests

Leave, complaint, room change, other. Two read paths on purpose:

```
Stream&lt;List&lt;HostelRequest&gt;&gt; watchAll(String collegeId)          // requests.viewAll
Stream&lt;List&lt;HostelRequest&gt;&gt; watchMine(String collegeId, String uid)  // requests.viewOwn
```

A student may only ever query their own, and the rules **enforce that the where-clause is present**. The author's name, registration number and room are copied onto the document so the staff queue renders a row without reading the user document for every entry.

`watchMine` is deliberately unordered in the query — an `orderBy` beside a `where` needs a composite index, and sorting twenty documents client-side is free.

The audit log

Two rules shape it.

Recording must never break the thing it is recording. Every `record` call swallows its own errors. A warden imposing a fine should not see it fail because the audit write did — the fine is the point, the log entry is bookkeeping.

Undo is generic. An entry carries the document path and the document's prior contents, so undo is "write `before` back, or delete if there was no before". There is no per-action switch statement that rots as modules are added.

Entries also carry an `undoCaveat` when restoring is only partial — for example, restoring a deleted user's profile cannot recreate their sign-in account.

Part V — The Admin Tools

Node scripts in `tools/`, run from your machine, using a **service-account key**. They are not part of the app and wardens never touch them.

Why they exist

A browser is untrusted, so the in-app importer creates accounts through the public sign-up door and gets throttled. A service-account key proves you own the project and unlocks the Admin SDK: no throttling, no secondary-app workaround, and — critically — **the ability to delete Auth accounts and enrollment records**, neither of which the app can do.

Setup

1. Firebase Console, then the gear icon, then **Project settings**
2. **Service accounts** tab, then **Generate new private key**
3. Rename the downloaded file to `serviceAccountKey.json` and put it in `tools/`
4. `cd tools && npm install`

> **This file is the master key to your entire Firebase project.** Anyone holding it can read, write or delete every account and every document, with no password, bypassing every security rule. It is gitignored — never email it, never paste it into a chat, never put it in a screenshot. If it leaks, revoke it immediately in the same Service accounts screen; deleting the file locally is not enough, because a committed file stays in git history forever.

The scripts refuse to run when the key's project does not match `.firebaserc`, and print both ids. A key from the wrong project authenticates perfectly and then reads a database your app never opens — every script reports success while the app goes on showing records you thought you deleted. That looks exactly like a caching bug and gets debugged as one for hours.

The scripts

Script	Purpose
`import-students.js`	Bulk-create from CSV, no throttling
`delete-students.js`	Remove students from **both** halves of Firebase
`fix-duplicate-emails.js`	Preflight for shared sibling addresses
`import-fines.js`	Bulk fine loading
`backup-firestore.js`	Snapshot collections to disk
`whois.mjs`, `countdocs.mjs`	Read-only inspection

What deletion actually clears

1. Room occupancy — `occupantUids` and hostel bed counters
2. Mess fee records
3. Fines
4. Requests

5. **Enrollment records**
6. Firestore profiles
7. **Firebase Auth sign-in accounts**

Steps 1 to 4 and 6 are also available in-app under Settings, Danger zone. Steps 5 and 7 are the ones **only this script can do** — 7 because the client SDK cannot delete other people's logins, and 5 because `firestore.rules` sets `allow delete: if false` on enrollments, so academic history is never deletable from a browser by design.

Hostels, rooms, college settings, roles, notices, office orders and the audit log are never touched. Those are the institution, not its intake.

```
node delete-students.js --from-csv ../students.csv          # preview
node delete-students.js --from-csv ../students.csv --commit
node delete-students.js --orphans --commit                  # logins with no profile
node delete-students.js --all-students --i-mean-it --commit # everything
```

A Super Admin is never deleted, under any combination of flags.

Part V — Testing

```
flutter analyze                          # ALWAYS before saying code is done
flutter test                             # Dart unit tests, no emulator
flutter test test/csv_import_test.dart   # one file
flutter test --plain-name "batch"        # one test by name
cd rules-tests &amp;&amp; npm test               # firestore.rules, via the emulator
```

`flutter analyze` is the single most useful thing you can run here. Much of this codebase was written without a compiler to hand, so treat a clean analyze as the **minimum bar, not a bonus**.

What is worth testing

The suite is 258 tests across 22 files, and it deliberately concentrates on **pure logic and known traps** rather than chasing coverage:

- `enrollment_helpers_test.dart` — batch to year, against the real cohorts on this campus rather than invented examples
- `stream_cache_test.dart` — the caching layer under every screen: opens once, replays correctly, survives the last listener leaving, replays errors
- `run_pooled_test.dart` — the throttle-narrowing behaviour, against a fake task
- `room_detail_dialog_test.dart` — pins the dialog/session trap by pumping with no scope above
- `csv_import_test.dart` — header aliases and derivation
- `firestore_indexes_test.dart` — that required indexes are declared

The pattern worth copying: **every bug that shipped got a test named after what it was**. `academic_session_test.dart` contains a case called "the promotion false alarm", which encodes an incident rather than an abstraction. Six months later that name explains itself.

Why `lib/utils/` is Firebase-free

```
/// Pure, synchronous helpers ... No Firebase calls here on purpose, so
/// every rule below can be unit tested without a database.
library;
```

Anything that can be a pure function should be, because pure functions are testable in milliseconds with no emulator, no mocks and no setup. Push logic down into `utils/`, keep `services/` as thin I/O, and your test suite stays fast enough that you actually run it.

Part V — Deployment

The command

```
./deploy-web.bat
```

Which does, in order: `flutter analyze`, `flutter test`, `flutter build web --release`, `firebase deploy --only hosting`. **Analyze and test run first and abort the deploy on failure.** A deploy script that skips the checks is a deploy script that ships broken builds at 11pm.

Hosting facts worth knowing

- HTTPS is automatic and mandatory. Plain HTTP receives a `301` to HTTPS.
- Firebase sets `Strict-Transport-Security` with `preload`, so after one visit a browser refuses to attempt HTTP for a year.
- SSL certificates are provisioned and renewed for free.
- Every site gets **two** domains: `<site>.web.app` and `<site>.firebaseapp.com`.
- The previous release keeps serving until a new one lands, and Hosting keeps release history for one-click rollback.

The multi-site trap

A project has a **default site** named after the project id, and may have additional named sites. Deploying with `"site": "homihostel"` in `firebase.json` updates **only that site** — the default site keeps serving whatever it last received.

Over time the default URL becomes an **old version of your app running against your live database**. That is a genuine hazard: stale logic, current data.

Do **not** solve it by disabling the default site. `authDomain` points at `<project-id>.firebaseapp.com`, and Firebase Auth serves its password-reset and email-verification handlers from `/__/auth/action` there. Disabling it breaks password resets application-wide. Redirect the default site instead.

Deploying rules is separate, and non-negotiable

`firebase deploy --only hosting` does **not** deploy rules. They are a separate target:

```
./deploy-rules.bat     # publishes, and records the deployed hash
./check-rules.bat      # says whether live rules match the file
```

Run the deploy immediately after editing `firestore.rules` — not at the end of the day, not before the next release. The gap between editing and deploying is where the confusing permission errors live.

If you cannot deploy — no credentials to hand — say so explicitly in whatever you hand over: a clearly marked note that the rules changed and must be deployed, naming what changed and who it affects. Silently editing the file and moving on has caused confusing permission errors in this project more than once.

Part VI — Every Trap This Project Hit

This is the most transferable chapter. Architecture ports poorly between projects; scar tissue ports well.

Flutter and Dart

`Session.of(context)` **throws inside a dialog.** `showDialog` pushes onto the root Navigator, so the dialog's context is a sibling of the dashboard rather than a descendant, and the lookup fails the moment it opens. `flutter analyze` cannot see it because it fails at runtime. Read the session in the calling page and pass what the dialog needs as constructor arguments. This shipped broken once.

A widget returning `Expanded` **or** `Spacer` **needs a bounded parent.** A card inside a `Wrap` has unbounded height; a vertical flex child there fails to lay out and silently **blanks the whole list**. Not an error — an empty screen. This happened once.

`ConstrainedBox` **is not const-constructible.** Its constructor asserts via a method call. Neither are `Container`, `AlertDialog` or `Tooltip`. You will reach for `const` and the compiler will disagree.

A `late final` **field that throws during initialisation rethrows on every access.** It does not retry. If the initialiser can fail, do not use `late final`.

Firestore

Reads are a budget, not free. Covered at length in Part III. The one-line version: never construct a query inside `build()`, whether it is a stream or a future.

Aggregations always hit the server. `count()` is not served from the local cache, so it costs a read every time even when the documents are cached. Memoise them.

Transactions forbid a read after a write. All `tx.get` calls must precede all `tx.update` calls. Runtime failure, not compile-time.

A batch cannot solve a race. Batches are atomic writes, not atomic read-modify-write. Only a transaction re-reads at commit time. Two wardens and one bed is a transaction.

`FieldValue.increment` **is atomic; read-then-write is not.** Under concurrency, read-modify-write loses updates silently.

`array-contains` **cannot look inside maps in an array.** If you need to query by a field of an array element, denormalise a flat array of just that field. This is why `studentUids` exists alongside `students[]`.

`orderBy` **beside a** `where` **needs a composite index.** For small result sets, sort client-side instead and keep your index configuration empty.

Firestore has no substring search. Prefix matching is possible with range queries; "contains" is not. Client-side filtering is the workaround, and it is why the user directory downloads the roster.

Documents cap at 1 MB. With base64 images that is a practical ceiling around 700 KB, since base64 inflates by roughly a third.

Web persistence is single-tab. A second tab gets no local cache and re-reads everything. If your usage looks inexplicably high, count the tabs.

Deleting a document does not delete its sub-collections. They survive as unreachable orphans that still cost storage. The Firebase Console offers a recursive delete; the SDK does not.

Firebase Auth

The client SDK can only delete the currently signed-in account. Everything else needs the Admin SDK. Plan for orphaned logins from day one, not after the first "email already in use".

`createUserWithEmailAndPassword` **signs the new user in**, logging your admin out. Use a secondary `FirebaseApp`.

Account creation from a browser is throttled project-wide. Tripping it affects real user sign-ins, not just your import. Back off aggressively and never tune the concurrency up to match a server-side script.

Auth persistence defaults to `LOCAL` **on web** — the session survives closing the browser. On shared machines that hands the next person a live session. Use `Persistence.SESSION`.

The domain

A bare year is not a session. `2026` instead of `2026-27` makes every batch fail to parse and backfill silently write nothing, while blaming 2,649 innocent student records. Validate the format where it is entered.

Comparing two totals is not a completeness check. The Academic Records page compared "students with a batch" against "enrolled this session" and warned when they differed. After a promotion they *correctly* differ — graduates leave the live roster but remain in the session they really were enrolled in. The page cried "roster incomplete" at the exact moment promotion had succeeded. Count what is actually missing instead:

```
int countMissing(Iterable&lt;String&gt; studentUids, Iterable&lt;String&gt; enrolledUids) {
  final enrolled = enrolledUids.toSet();
  return studentUids.where((uid) =&gt; !enrolled.contains(uid)).length;
}
```

A status field only one screen respects is worse than none. `isGraduated` is checked in one place; four other screens still count graduates as current students. It looks like it works.

A new collection is not automatically covered by your cleanup. `enrollments` was added long after `delete-students.js` was written, so a full wipe left thousands of orphaned records that made Academic Records report students against an empty roster. When you add a collection, immediately ask what deletes it.

Bulk operations are the real quota cost. Two full wipe-import-promote cycles in one day exhausted a 50,000-read budget. No amount of read optimisation touches this.

Operational

Rules on disk do nothing. Only the deployed version has any effect. Deploy immediately after editing, and keep a `check-rules` command that tells you whether they match.

Testing as Super Admin exercises no permission branches. `hasPerm` short-circuits. Create a Warden and a Student.

A service-account key from the wrong project authenticates perfectly and then operates on a database your app never opens. Every script reports success; the app shows records you thought you deleted. Make your scripts compare the key's project id against `.firebaserc` and refuse to run on a mismatch.

An eternal spinner is indistinguishable from a hang. The dashboard gates on `if (!counts.hasData)` with no error branch, so a quota-exhausted aggregation spins forever with no explanation. **Always handle** `hasError`**.** A failed future is not a slow future.

Part VI — Build It Again From Scratch

The recipe, in the order that avoids rework.

1. Model the domain before writing any UI

Write down your entities and — more importantly — **what each one belongs to**. The single best decision in this project was recognising that *year belongs to a student-in-a-session*, not to a student. That fell out of asking "if I overwrite this every July, what have I destroyed?"

Ask that question of every field. Anything whose old value has future value is history and needs its own document.

2. Design permissions before roles

List every *action* the system can perform. Group them `module.action`. Do **not** start from job titles — job titles are data, actions are code. Roles become named bundles of permissions, stored in the database, editable by an administrator.

This single inversion is what lets a college invent "Block Supervisor" without a code change.

3. Set up Firebase and write the rules first

Rules written after the fact are rules written to match whatever the app happens to do, which is the wrong direction. Write them alongside the model, deploy them immediately, and test the **denials** in the emulator. A rule that allows everything passes all your permission tests.

4. Build the shell, the session and the registry

One dashboard. One navigation list with `requires:` per entry. One `Session` in an `InheritedWidget`. Get this right on day one; retrofitting a permission system into a built app is brutal.

5. One service per module, and never a query in a widget

Write the caching layer **early**. Retrofitting `CachedStreamPool` into thirty screens is a large, error-prone change; having it from the start costs nothing.

6. Feature by feature, three files at a time

One nav entry, one page, one rules block. If a feature needs more than that, the architecture is fighting you.

7. Write the tools before you need them

Bulk import, bulk delete, backup. You will need all three during your first real data load, and you will need them urgently, which is the worst time to write them.

8. Measure reads from the first day

Open the Firestore usage graph after building each screen. A screen that costs 2,650 reads is not a problem you notice in development with twelve test records — it is a problem you notice in production with a real roster, on the day it stops working.

What I would do differently

Force a password change on first login. The derived-password scheme is the one genuine security weakness in the system.

Budget for Blaze from the start, even if you never leave the free tier in practice. Cloud Functions solve account deletion, enrollment cleanup and every other privileged operation properly, and the free allowance on Blaze is generous. Most of the awkwardness in the tooling exists to work around not having them.

Decide what "archived" means across the whole app at once. Graduation, deactivation and deletion are three different kinds of gone, and every screen needs to agree on how each is treated.

Put images in a sub-collection from the beginning, even without Cloud Storage. `officeOrders/{id}/media/photo` costs nothing extra to design up front and saves a 35 MB page later.

Validate every user-entered format at entry. The session-format incident cost hours and was preventable with one regex.

Appendix A — Command Reference

```
# Development
flutter analyze                          # before saying anything is done
flutter test                             # all Dart tests
flutter test test/csv_import_test.dart   # one file
flutter test --plain-name "batch"        # one test by name
flutter run -d edge                      # web, the primary target
flutter run -d windows                   # desktop

# Rules
./deploy-rules.bat                       # publish + record hash
./check-rules.bat                        # do live rules match the file?
cd rules-tests &amp;&amp; npm test               # emulator assertions

# Deploy
./deploy-web.bat                         # analyze + test + build + deploy

# Admin tools (from tools/, needs serviceAccountKey.json)
node import-students.js sheet.csv                    # dry run
node import-students.js sheet.csv --commit
node fix-duplicate-emails.js ../sheet.csv            # preflight
node delete-students.js --orphans --commit           # logins with no profile
node delete-students.js --all-students --i-mean-it --commit
node backup-firestore.js
```

Appendix B — The Permission Catalogue

Module	Permissions
Users	`users.view`, `users.create`, `users.edit`, `users.delete`
Roles	`roles.view`, `roles.manage`
Hostels	`hostels.view`, `hostels.manage`, `allotment.manage`
Mess	`mess.view`, `mess.manage`
Requests	`requests.create`, `requests.viewOwn`, `requests.viewAll`, `requests.approve`
Fines	`fines.viewOwn`, `fines.viewAll`, `fines.manage`
Office orders	`officeOrders.view`, `officeOrders.manage`
Academic	`academic.view`, `academic.manage`
Mess fees	`fees.viewOwn`, `fees.viewAll`, `fees.manage`
Notices	`notices.view`, `notices.manage`
Self-service	`self.room`
System	`settings.manage`

Appendix C — Glossary

Batch — the academic session in which a student's Year 1 began, written `2021-22`. The input to every year calculation.

Session — an academic year, written `2026-27`. Set in System Settings; scopes Academic Records.

Enrollment — one student's record for one session, at `{uid}_{session}`. Holds the year. The unit of academic history.

Backfill — creating enrollments from `batch` + session. The one-time bootstrap.

Promotion — advancing everyone by reading last session's year and adding one. Annual.

Graduated — a status on the profile, set when promotion would exceed the course length. No next-session record is written; nothing is deleted.

Orphan — a Firebase Auth login with no profile document behind it. Causes "email already in use" on re-import. Only the Admin SDK can remove one.

Synthetic email — `{registrationNo}@homihostel.local`, generated so students can sign in with a roll number. Non-routable; has no inbox.

Derived password — the registration number, padded to six characters. The starting password for a bulk-imported student.

Spark plan — Firebase's free tier. 50,000 reads, 20,000 writes and 20,000 deletes per day; no Cloud Functions; no Cloud Storage.

Closing

You said you did not feel good about how much of this was abstraction you could not see into. That instinct is the right one, and it is worth naming what it actually is: the difference between having a working system and understanding one.

The gap is smaller than it feels. The reason this codebase is dense is not that it is clever — it is that almost every non-obvious line is the scar of a specific failure. The transaction exists because two wardens can click the same bed. The stream cache exists because a roster burned a day's quota in nineteen screen opens. The `excludes` field exists because a Super Admin saw a page about a room they will never have. None of that is knowledge you could have had in advance. It is knowledge the project produced, and it is now yours.

What transfers to the next thing you build is not this architecture. It is the habits underneath it: model what belongs to what before writing a screen; make permissions actions rather than job titles; write the security layer alongside the data model rather than after it; measure the expensive resource from day one; and when something breaks, write the test that names the incident.

The vision was yours. The domain knowledge — how an office order really works, that rooms reset every July, that a caretaker and a warden are different jobs — was never in any documentation and could only have come from you. Build the next one knowing that part was always the harder half.